

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 Обзор предметной области	3
2 Общие определения	5
3 Перекрытия	7
4 Каскадное произведение автоматов	10
5 Алгоритм	16
6 k -СЛТЯ	17
6.1 Получение нормальной формы СЛТ языка	18
6.2 Шаблоны, описываемые СЛТ языками	21
7 Распознавание перекрытий на основе антиязыков шаблонов .	24
8 Результат научно-исследовательской работы	25
Список литературы	26

ВВЕДЕНИЕ

Основная конструкция в Рефале — сопоставление с образцом. Будучи при этом нетипизированным языком в привычном понимании, Рефал-программы могут демонстрировать потрясающую выразительность. Но с большими возможностями появляется большая ответственность: отсутствие типизации — это источник потенциальных ошибок, а полный статический анализ шаблонов в левых частях функций в общем случае является неразрешимой задачей (в диалектах языка, допускающих использование условий, в условиях могут быть произвольные вычисляемые функции).

Одним из видов семантических ошибок является (множественное) перекрытие предложения — ситуация, когда некоторое предложение функции недостижимо при любом значении аргумента. Такие предложения почти всегда появляются в результате ошибки при написании кода, при этом их трудно отловить как на этапе написания программы, так и во время ее выполнения. Первое обусловлено тем, что перекрытия могут быть множественными, т.е. одно предложение экранируется несколькими другими, при этом в функции эти предложения могут идти не подряд и визуально обнаружить такие перекрытия крайне сложно. Второе же обуславливается тем, что экранируемые предложения де-факто являются мертвым кодом, а код, который не исполняется, не порождает ошибок времени выполнения.

1 Обзор предметной области

Язык программирования Рефал имеет довольно большую историю, начавшуюся в 1968 г., когда появилась первая реализация Рефала. Рефал имеет множество диалектов, но все они, по сути, являются надстройками над так называемым базисным Рефалом. Базисный рефал в некотором смысле является самым простым диалектом языка. Программы на базисном Рефале состоят из функций, каждая функция задается парами из шаблона в левой части и выражения в правой.

Единственный тип данных в Рефале — объектное выражение. Это выражение, состоящее из атомарных символов (символ, число) и скобок, при этом скобки правильно вложены. Шаблон в левой части функции — это выражение, в котором, помимо скобок и атомов, могут встречаться переменные. Переменные в шаблоне могут быть трех типов: *e*-переменные, *s*-переменные и *t*-переменные. *e*-переменные могут сопоставляться с любым объектным выражением, *s*-переменные сопоставляются с атомом, *t*-переменные сопоставляются либо с атомом, либо с выражением в скобках. Каждая переменная обладает именем, при этом, если в одном шаблоне есть несколько переменных с одним именем, то при подстановке они заменяются на разные значения.

Основной механизм работы Рефала — сопоставление с образцом и переписывание выражений. При сопоставлении с образцом, шаблон в левой части функции сопоставляется с объектным выражением в правой части. Если сопоставление прошло успешно, переменным присваиваются конкретные значения, затем вычисляется выражение в правой части. Сопоставление происходит сверху вниз, если какое-то выражение не сопоставилось ни с одним предложением, это приводит к ошибке времени выполнения.

В листинге 1 функция `IsPal` проверяет, является ли переданное ей выражение, состоящее только из атомов, палиндромом.

Листинг 1 — Пример функции на базисном Рефале

```
1 IsPal {
2   s.x e.x s.x = <IsPal e.x>;
3   = True;
4   s.oneChar = True;
5   e.otherwise = False;
6 }
```

Будучи преимущественно объектом исследования в области суперкомпиляции, Рефал-программы можно исследовать и в контексте статического анализа. Статический анализ программ на Рефале представляет интерес по нескольким причинам. Во-первых, специфика языка — отсутствие явных циклов и присваиваний, опора на сопоставление с образцом и рекурсию — приводит к тому, что многие классические методы анализа, разработанные для императивных языков, неприменимы напрямую и требуют адаптации. Во-вторых, динамическая природа объектных выражений и наличие переменных, способных сопоставляться с выражениями произвольной длины (в первую очередь *e*-переменных), порождает специфические задачи: например, оценку возможных форм результата функции или определение множества выражений, с которыми может быть вызвана та или иная функция.

2 Общие определения

Определение 1 (Алфавит). Алфавитом Σ будем называть конечное множество символов.

Будем обозначать алфавиты заглавными греческими буквами.

Определение 2 (Связанные со строками понятия). Строкой $w = w_1w_2 \dots w_n$ будем называть конечную последовательность символов из Σ .

Длиной строки w будем называть число n , обозначаемое как $|w|$.

Пустым словом будем называть строку длины 0, обозначаемую как ε .

i -тый символ строки w будем обозначать как $w[i]$.

Строки будем обозначать строчными латинскими буквами.

Определение 3 (Обращение строки). Для строки $w = w_1w_2 \dots w_n$ будем называть ее обращением строку $w^R = w_n \dots w_2w_1$. Для множества строк $W = \{w_1, w_2, \dots\}$ будем называть его обращением множество $M^R = \{w_n^R, w_2^R, \dots\}$.

Определение 4 (Итерация Клини). Множеством строк Σ^* будем называть множество всех конечных строк над алфавитом Σ . $\varepsilon \in \Sigma^*$. Множество $\Sigma^* \setminus \{\varepsilon\}$ обозначим Σ^+ .

Определение 5 (Сокращенная запись конкатенации). Конкатенацией строк $u = u_1u_2 \dots u_n$ и $v = v_1v_2 \dots v_m$ будем называть строку $u \cdot v = u_1u_2 \dots u_nv_1v_2 \dots v_m$.

Если $M \subset \Sigma^*$ — некоторое множество строк и $w \in \Sigma^*$, то будем обозначать как Mw множество $\{mw \mid m \in M\}$. Аналогично для двух множеств строк M_1, M_2 будем обозначать M_1M_2 множество $\{m_1m_2, m_i \in M_i\}$.

Определение 6. Множество $\Sigma^* \setminus W$ будем обозначать $\overline{W}$ и называть дополнением языка W .

Множества строк будем обозначать заглавными латинскими буквами.

Определение 7 (Строковый морфизм). Строковый морфизм $\phi: \Sigma^* \rightarrow \Gamma^*$ — это функция, которая каждому слову $w \in \Sigma^*$ ставит в соответствие слово $\phi(w) \in \Gamma^*$, сохраняющая конкатенацию, т.е. $\forall w_1, w_2 \in \Sigma^* \phi(w_1w_2) = \phi(w_1)\phi(w_2)$. Строковый морфизм однозначно задается набором значений на символах алфавита Σ .

Если для некоторого $\sigma \in \Sigma$ не задано явным образом отображение, будем считать, что σ отображается в себя. В таком случае будем опускать этот факт в записи областей определения и значений морфизма.

Все морфизмы в данной работе — строковые, поэтому будем опускать слово "строковый". Будем обозначать морфизмы строчными греческими буквами.

Пример 1. Пусть $\Sigma = \Gamma = \{a, b, c\}$. Морфизм, меняющий местами a и b , можно задать следующим образом: $\phi: \{a, b\} \rightarrow \{a, b\}$, при этом

$$\phi(a) \mapsto b;$$

$$\phi(b) \mapsto a.$$

В таком способе записи неявно задано $\phi(c) = c$.

Определение 8 (синтаксическая конгруэнция). Синтаксической конгруэнцией языка L будем называть отношение эквивалентности $\sim_L$ такое, что

$$u \sim_L v \iff \forall x, y \in \Sigma^* (xuy \in L \iff xvy \in L)$$

Класс эквивалентности некоторого x в фактор-структуре будем обозначать, как $[x]$

Определение 9 (Синтаксический моноид). Фактор-множество $\Sigma^* / \sim_L$ с операцией конкатенации ($[x][y] = [xy]$) — синтаксический моноид языка L .

Определение 10 (синтаксическая полугруппа). Фактор-множество $\Sigma^+ / \sim_L$ с операцией конкатенации ($[x][y] = [xy]$) — синтаксическая полугруппа языка L .

Для $n \in \mathbb{N}$ примем $\overline{1, n} = \{1, 2, \dots, n\}$.

3 Перекрытия

Определение 11 (Язык правильно вложенных слов). Пусть Σ_{call} , Σ_{ret} и Σ_{int} — непесекающиеся алфавиты. Язык правильно вложенных слов $\mathcal{W}$ — это наименьшее множество, которое генерируется следующими конструкторами:

$$\begin{aligned} \varepsilon &\in \mathcal{W} && \text{(пустое слово)} \\ \gamma &\in \mathcal{W} && \text{для каждого } \gamma \in \Sigma_{\text{int}} \quad \text{(внутренний символ)} \\ w_1 \cdot w_2 &\in \mathcal{W} && \text{если } w_1, w_2 \in \mathcal{W} \\ \langle_c w \rangle_r &\in \mathcal{W} && \text{если } w \in \mathcal{W}, \langle_c \in \Sigma_{\text{call}}, \rangle_r \in \Sigma_{\text{ret}} \end{aligned}$$

Выражением будем называть слова языка правильно вложенных слов $\mathcal{W}_{\text{expr}}$, порожденного алфавитами $\Sigma_{\text{call}} = \{(\,, \,)\}$, $\Sigma_{\text{ret}} = \{)\}$ и $\Sigma_{\text{int}} = \Delta$, где Δ — счетное множество символов. Абстрактным шаблоном без повторных переменных будем называть слова языка правильно вложенных слов $\mathcal{W}_{\text{pat}}$, порожденного алфавитами $\Sigma_{\text{call}} = \{(\,, \,)\}$, $\Sigma_{\text{ret}} = \{)\}$ и $\Sigma_{\text{int}} = \{e, s, t\}$. Введем также понятие шаблона с пронумерованными переменными: каждому шаблону $p \in \mathcal{W}_{\text{pat}}$ поставим в соответствие слово над алфавитом $\Delta_{\mathbb{N}} = \{(\,, \,)\} \cup \{e, s, t\} \times \mathbb{N}$ такое, что все вхождения символов e, s, t в p в шаблоне с пронумерованными переменными заменяются на буквы из $\{e, s, t\} \times \mathbb{N}$, причем i -тому вхождению символа e в p соответствует буква (e, i) (обозначим e_i). Аналогично для s и t . Скобки $(\,, \,)$ остаются на месте. Для шаблона p будем обозначать соответствующий ему шаблон с пронумерованными переменными $p_{\mathbb{N}}$.

Замечание 1. Формальное описание абстрактного шаблона без повторных переменных соответствует таким шаблонам языка Рефал, в которых все переменные имеют различные имена и атомарные символы не встречаются вовсе. Например, для диалекта Рефал5- λ , поддерживающего безымянные переменные, из абстрактного шаблона можно получить обычный, заменив все символы e абстрактного шаблона на символы $e _$ шаблона Рефал5- λ .

Соответствие переменных множествам их допустимых значений было приведено в обзоре базисного Рефала, теперь же формализуем это соответствие с учетом, что рассматриваются только абстрактные шаблоны без повторных переменных. Будем говорить, что выражение w допускает сопоставление с образцом

p , если существует строковый морфизм $\phi: \Delta_{\mathbb{N}}^* \rightarrow \mathcal{W}_{\text{expr}}$, действующий следующим образом:

$$\begin{aligned}\phi((s, i)) &\mapsto \gamma \in \Sigma_{\text{int}} \quad \forall i \in \mathbb{N}; \\ \phi((t, i)) &\mapsto \gamma \in \Sigma_{\text{int}} \text{ или } \phi((t, i)) \mapsto (v), \quad v \in \mathcal{W}_{\text{expr}} \quad \forall i \in \mathbb{N},\end{aligned}$$

при этом $\phi(p_{\mathbb{N}}) = w$.

Определение 12 (Язык шаблона). Языком шаблона $p \in \mathcal{W}_{\text{pat}}$ будем называть множество всех выражений $w \in \mathcal{W}_{\text{expr}}$, которые допускают сопоставление с p . Обозначать это множество будем как $\mathcal{L}(p)$.

Каждая функция Рефала задается последовательностью предложений, каждое из которых в свою очередь содержит шаблон в левой части. Для некоторой функции f , содержащей n предложений, будем обозначать i -тый по счету (считая сверху вниз) шаблон предложения как $f.p_i$. Последовательность всех шаблонов функции $\langle f.p_1, f.p_2, \dots, f.p_n \rangle$ будем обозначать как **pat** f .

Определение 13 (Перекрытие). Будем говорить, что шаблон p *перекрывается* или *экранируется* шаблонами $p_1, p_2, \dots, p_k$, если

$$\mathcal{L}(p) \subseteq \bigcup_{i=1}^k \mathcal{L}(p_i).$$

Будем говорить, что перекрытие (экранирование) *множественное*, если $k > 1$.

Итак, после введения всех необходимых понятий, мы можем перейти к определению задачи распознавания перекрытия:

Определение 14 (Задача распознавания перекрытия). Задача распознавания перекрытия в функции f с n предложениями заключается в том, чтобы для каждого шаблона $p_i \in \mathbf{pat} f$, $i \in \overline{1, n}$ найти минимальное по включению множество $I = \{i_1, i_2, \dots, i_k\} \subseteq \overline{1, i-1}$ такое, что p_i перекрывается шаблонами $f.p_{i_1}, f.p_{i_2}, \dots, f.p_{i_k}$, или показать, что такого множества не существует.

Отметим, что решение задачи распознавания перекрытия неоднозначно, так как для одного и того же шаблона можно найти несколько минимальных по включению множеств индексов таких, что соответствующие им шаблоны перекрывают

искомый. Пример такого случая приведён в листинге 2. Предложение 5 перекрывается предложениями 3 и 4, а также предложением 1. При этом множества $\{3, 4\}$ и $\{1\}$ несравнимы по включению.

Листинг 2 — Пример неоднозначного экранирования предложений Рефал-функции

```
1 F {
2     (t._ t._) e._ = ...;           /* 1 */
3     e._ (s._ e._ s._) e._ = ...;   /* 2 */
4     e._ s._ (t._) = ...;           /* 3 */
5     e._ (e._) (t._) = ...;         /* 4 */
6     (t._ t._) t._ (t._) = ...;     /* 5 */
7 }
```

4 Каскадное произведение автоматов

Данная работа ставит целью на решении специального случая задачи распознавания перекрытий, когда шаблоны функции имеют особый вид, позволяющий решать задачу относительно эффективно. О виде таких шаблонов и, непосредственно, об алгоритме детекции перекрытий будет рассказано в следующих разделах.

Регулярное представление шаблона представляет особый интерес, поскольку, в сравнении с контекстно-свободными языками (в частности, языками с магазинных автоматов, управляемых входом), коими являются $\mathcal{W}_{\text{expr}}$ и $\mathcal{W}_{\text{pat}}$, регулярные языки в каком-то смысле более просты для анализа. Более того, впоследствии мы перейдем к еще более узкому классу языков, являющимся подклассом регулярных.

Рассмотрим основную теоретическую конструкцию, позволяющую извлекать регулярное представление шаблонов — каскадное произведение автоматов:

Определение 15 (Каскадное произведение автоматов). Пусть $A_1 = \langle Q_1, \Sigma, \delta_1, q_{0_1}, F_1 \rangle$ и $A_2 = \langle Q_2, \Sigma \times Q_1, \delta_2, q_{0_2}, F_2 \rangle$ — детерминированные конечные автоматы. Каскадным произведением $A_1 \circ A_2$ называется ДКА $\langle Q, \Sigma, \delta, q_0, F \rangle$, где

$$\begin{aligned} Q &= Q_1 \times Q_2, \\ q_0 &= (q_{0_1}, q_{0_2}), \\ F &= F_1 \times F_2, \\ \delta((q_1, q_2), a) &= (\delta_1(q_1, a), \delta_2(q_2, (a, q_1))). \end{aligned}$$

Ту же конструкцию можно описать как систему из двух автоматов, работающих параллельно на одном и том же входном слове, но с односторонней зависимостью: оба автомата синхронно читают входное слово $w \in \Sigma^*$, при этом первый автомат A работает автономно над алфавитом Σ , а второй автомат, помимо символа в w , использует текущее состояние первого автомата q , т.е. работает над алфавитом $\Sigma \times Q_1$.

Основная сложность получения регулярного вида — скобочная структура шаблона. Введем функцию глубины вложенности на элементах языка правильно

ВЛОЖЕННЫХ СЛОВ:

$$d(w, i) = \begin{cases} 0, & i = 0 \\ d(w, i - 1) + 1, & w[i] \in \Sigma_{\text{call}} \\ d(w, i - 1) - 1, & w[i] \in \Sigma_{\text{ret}} \\ d(w, i - 1), & w[i] \in \Sigma_{\text{int}} \end{cases}$$

Тогда максимальным уровнем вложенности некоторого слова w будет

$$\max d(w) = \max_{i \in \overline{1, |w|}} d(w[i]).$$

Идея получения регулярного представления шаблона заключается в том, чтобы итеративно понижать максимальный уровень вложенности шаблона путем последовательного применения каскадного произведения и изменении алфавита каждого нового элемента каскада. Основной автомат распознает шаблоны уровня вложенности 0, делегируя распознавание шаблонов уровня вложенности 1 второму подчиненному автомату. Тот, в свою очередь, задачу распознавания шаблонов уровня вложенности 2 предоставляет своему подчиненному автомату, и так далее. Заметим, что каскад конечен, т.к. конечна величина $\max d(w)$ для любого w .

Введем искомый алфавит $\Gamma_0 = \{b, s, (,)\}$. b является мета-символом, соответствующим словам $\mathcal{W}_{\text{expr}} \ni w = (v)$, $v \in \mathcal{W}_{\text{expr}}$, s также является мета-символом, соответствующим словам $\mathcal{W}_{\text{expr}} \ni w = \gamma \in \Sigma_{\text{int}}$. Будем рассматривать регулярные выражения r над алфавитом Γ_0 . Так как эти выражения, по сути, задают некоторый язык шаблонов, во избежание путаницы, будем обозначать язык регулярного выражения r как $R(r)$. Для слова $u \in R(r)$ поставим ему в соответствие слово $u_{\mathbb{N}}$ над $\Gamma_{\mathbb{N}} = \{(,)\} \cup \{b, s\} \times \mathbb{N}$ с пронумерованными переменными, аналогично тому, как это было сделано для шаблонов $p \in \mathcal{W}_{\text{pat}}$ и $p_{\mathbb{N}}$. Будем говорить, что w допускает сопоставление с r , если

$$\exists u \in R(r) \exists \phi: \{b, s\} \times \mathbb{N} \rightarrow \mathcal{W}_{\text{expr}} \mid \phi(u_{\mathbb{N}}) = w,$$

при этом

$$\begin{aligned}\phi((b, i)) &\mapsto (w) \in \mathcal{W}_{\text{expr}} \forall i \in \mathbb{N}; \\ \phi((s, i)) &\mapsto \gamma \in \Sigma_{\text{int}} \forall i \in \mathbb{N};\end{aligned}$$

Множество слов $w \in \mathcal{W}_{\text{expr}}$, допускающих сопоставление с r , будем обозначать как $\mathcal{L}(r)$. Любой шаблон $p \in \mathcal{W}_{\text{pat}}$ можно представить в виде регулярного выражения r над алфавитом Γ так, что $\mathcal{L}(p) = \mathcal{L}(r)$. Данный факт следует непосредственно из определения самих e, t, s -символов, ниже представлено соответствие между символами и регулярными выражениями:

- e -символ соответствует выражению $(b|s)^*$;
- t -символ соответствует выражению $(b|s)$;
- s -символ соответствует выражению s .

Для распознавания перекрытий важно учитывать структуры подслов шаблонов каждого уровня вложенности. Поэтому на каждом этапе каскад для уровня вложенности k должен учитывать все возможные подслова шаблонов уровня вложенности k . Опишем общий алгоритм и рассмотрим пример. Сначала в шаблонах выделяются подслова максимального уровня вложенности $m = \max d(w)$. Будем называть подслова шаблона уровня вложенности k k -шаблонами. Пусть N_k есть количество различных k -шаблонов. Будем подразумевать, что всем различным k -шаблонам поставлены в соответствие индексы из $\overline{1, N_k}$. Для всех m -шаблонов строится аннотированный ДКА

$$A = (Q, \Gamma_0, \delta: \Gamma_0 \times Q \rightarrow Q, q_0 \in Q, \alpha: Q \rightarrow \overline{1, N_m})$$

такой, что аннотация $\alpha(q)$ соответствует множеству индексов распознаваемых m -шаблонов. Все ДКА имеют схожий вид: они состоят из автомата-лифта, который отслеживает уровень вложенности, и автомата-распознавателя, который по достижении нужного уровня вложенности, распознает соответствующие ему шаблоны. Следующий автомат каскада состоит из лифта на $m - 1$ уровень и распознает шаблоны уровня вложенности $m - 1$. Элементы его алфавита теперь суть пары из $\Gamma_0 \times Q$. Автомат полностью игнорирует подчиненный автомат более высокого уровня вложенности, пока не потребуется распознать шаблон уровня вложенности m , по скобке основной автомат всегда переходит в ожидающее состояние, а выход

осуществляется по закрывающей скобке в зависимости от состояния подчиненного автомата. Рассмотрим пример функции, приведённой в листинге 3.

Листинг 3 — Пример функции на базисном Рефале

```

1 f {
2   t (e (s) s) = ...
3   ((t) e s) s = ...
4 }
```

Для примера в листинге 3 шаблонами уровня вложенности 2 будут s, t ; уровня 1 — $e(s)s, (t)es$; без скобочной вложенности — $t(e(s)s), ((t)es)s$. На рисунках 1, 2 и 3 представлен каскад полных автоматов слева и упрощённых автоматов-распознавателей справа, работающих над алфавитами, которые не содержат скобок. Для лучшего визуального восприятия будем злоупотреблять нотацией. Красным помечены состояния автомата-лифта, пунктирными границами выделены состояния автомата-распознавателя. Синим цветом — состояния, которые сохраняются при преобразовании полного автомата в упрощённый. В некоторых элементах каскада, в которых присутствует лифт уровня k , в состояниях, откуда нет перехода по какой-либо паре с открывающей скобкой в качестве первого символа, будем подразумевать наличие перехода на уровень $k - 1$ лифта. В автоматах, работающих в комбинированном алфавите $\Sigma \times \Gamma_i$, если переход возможен по паре с любым вторым символом $\gamma \in \Gamma_i$, будем опускать второй символ пары. Обозначение $X; Y, Z, \dots$ означает наличие перехода по парам $(X, Y), (X, Z), \dots$. Обозначение $X; \overline{Y, Z, \dots}$ означает наличие перехода по всем парам (X, Q) , где $Q \neq Y \wedge Q \neq Z \wedge \dots$.

Заметим, что по построению автомат для k -шаблонов не различает состояния автомата $(k + 1)$ -шаблонов с одинаковыми аннотациями. Это замечание позволяет упростить структуру распознавателя для шаблонов уровня вложенности n : Введем на множестве состояний подчиненного автомата-распознавателя отношение эквивалентности $\sim_\alpha$ такое, что $q_1 \sim_\alpha q_2$ тогда и только тогда, когда $\alpha(q_1)$ и $\alpha(q_2)$, тогда вместо алфавита $\Gamma_0 \times Q$ можно использовать алфавит $\{s, (,)\} \cup Q / \sim_\alpha$. Элементы $Q / \sim_\alpha$ будем обозначать b_i . Переходы упростим сле-

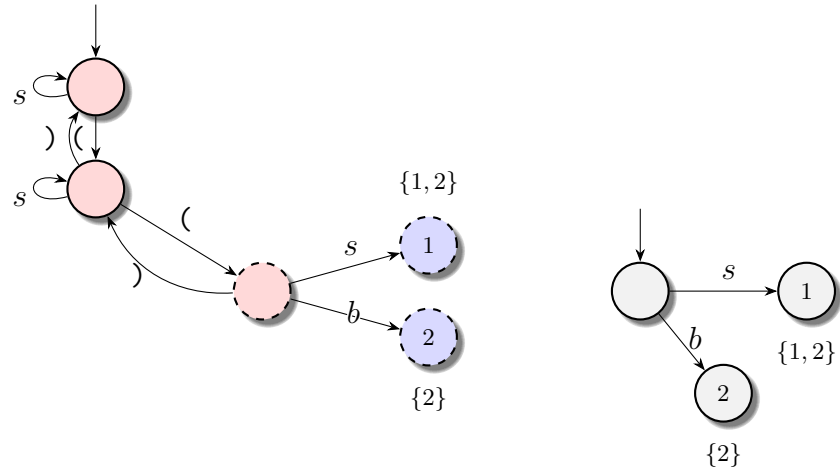


Рис. 1 — Аннотированный распознаватель для шаблонов уровня вложенности 2 (слева). Справа — тот же распознаватель без лифта над алфавитом $\{b, s\}$.

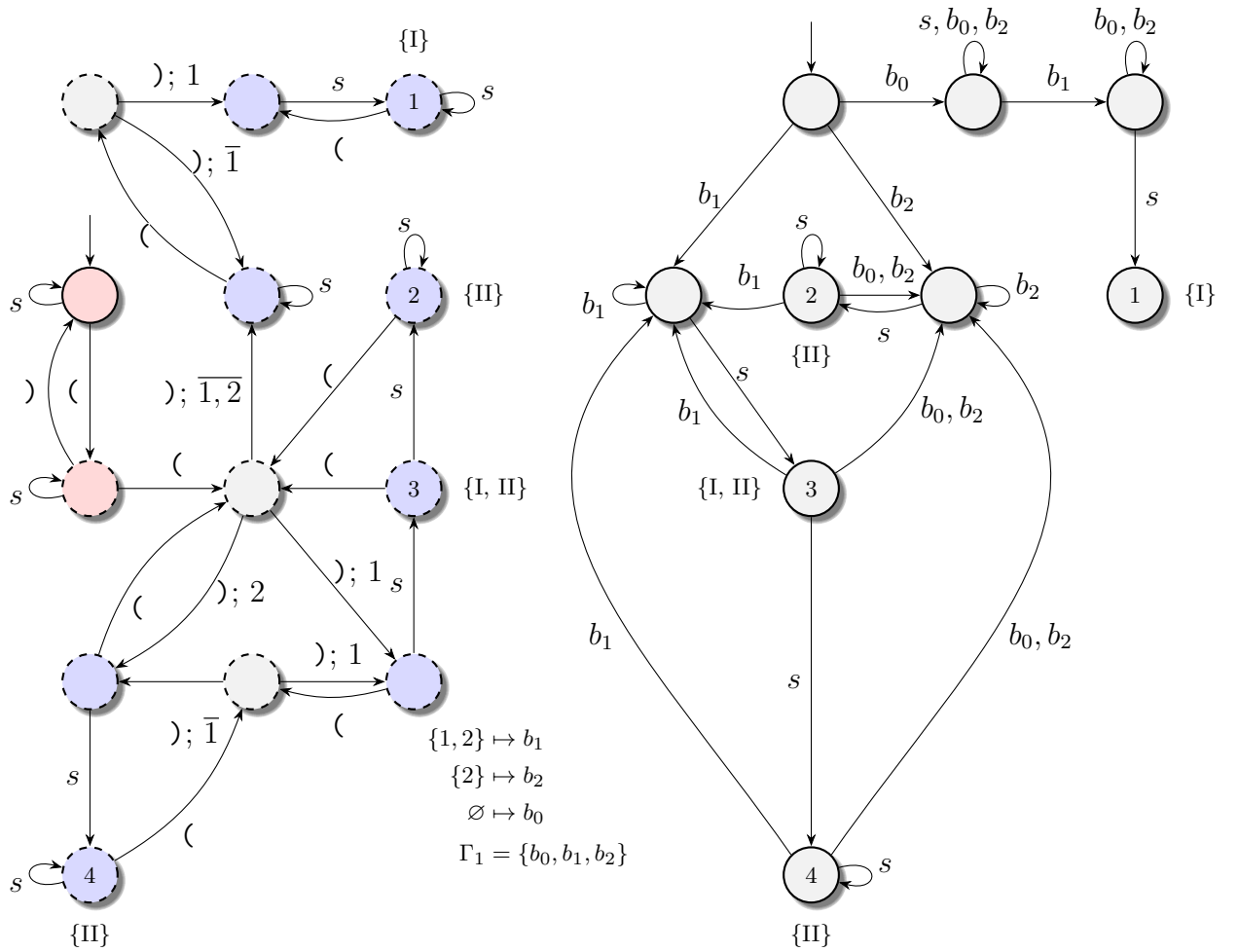


Рис. 2 — Пример аннотированного ДКА (слева) для 1-шаблонов. Справа — упрощённый распознаватель над алфавитом $\{s\} \cup Q_2 / \sim_\alpha$.

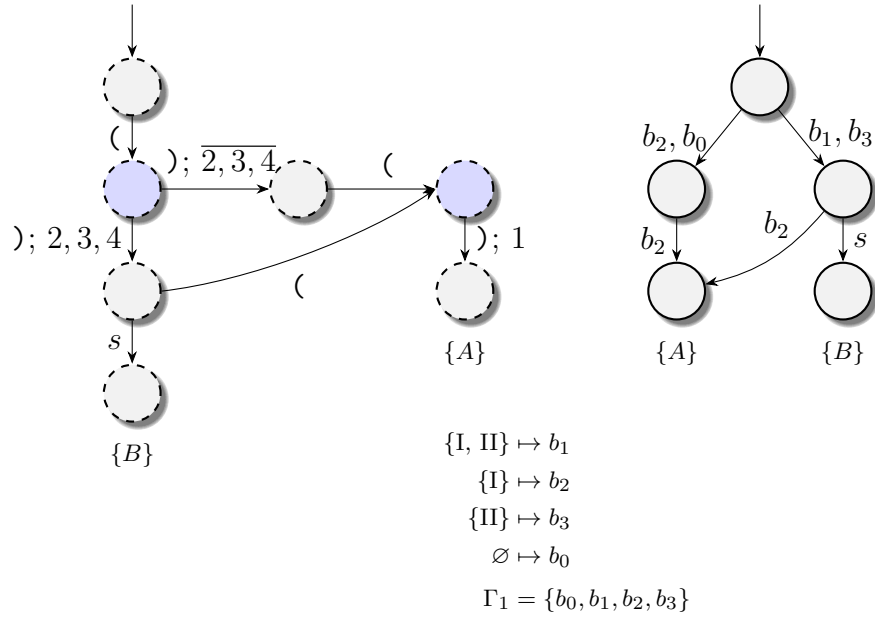


Рис. 3 — Последний элемент каскада уровня 0 (слева). Справа — упрощенный автомат-распознаватель над алфавитом $\{s\} \cup Q_1 / \sim_\alpha$

дующим образом — вместо перехода по скобке в состояние ожидания и выхода в зависимости от состояния подчиненного автомата, можем осуществлять переход сразу по b_i . Не считая лифта, получили новый автомат-распознаватель над алфавитом $\{s\} \cup Q / \sim_\alpha$. Обратим внимание, что для автомата 0-шаблонов лифт не нужен и де-факто автомат построен над алфавитом $\{s\} \cup Q_1 / \sim_\alpha$. Заметим, что из такого автомата можно получить автоматы, распознающий каждый шаблон с индексом i по отдельности — для этого достаточно в искомом автомате сделать принимающими те состояния, для которых справедливо $\alpha(q) \ni i$.

5 Алгоритм

Определить уровень $d = \max d(w)$ для всех шаблонов $p \in P$. Ввести начальный алфавит $\Gamma_0 = \{b, s\}$. Над алфавитом Γ_0 построить аннотированный ДКА A_d для шаблонов уровня вложенности d . $\Gamma_1 = \{s\} \cup Q_1 / \sim_\alpha$. В искомым шаблонах заменить все подслова уровня вложенности d на регулярное выражение $r = (b_{I_1}|b_{I_2}|\dots|b_{I_n})$, $\forall i \in \overline{1, n} \ b_{I_i} \in Q_1 / \sim_\alpha$, причем b_j является вариантом альтернативы r тогда для подслова под номером m тогда и только тогда, когда $\alpha(q_j) \ni m$. 1. Перевести все шаблоны

Выше было показано, как по шаблону p построить регулярное выражение r . Алгоритм заключается в определении подслов шаблонов максимального уровня вложенности m и построении аннотированного ДКА для них. Далее вводится новый алфавит $\Gamma_i = \{s, (,)\} \cup Q_{i+1} / \sim$, где Q_i — множество состояний ДКА для шаблонов уровня вложенности i .

6 k -СЛТЯ

Существуют подклассы регулярных языков, допускающие описание в некотором смысле более простое, чем с помощью ДКА. Одним из таких классов, представляющих непосредственный интерес, являются строго локально k -тестируемые языки (СЛТ языки).

Определение 16 (k -СЛТЯ). Язык $\mathcal{L} \subseteq \Sigma^+$ называется строго локально k -тестируемым (k -СЛТЯ), если существуют множества $P, S, F \subseteq \{w \in \Sigma^+ \mid |w| \leq k\}$ и $W \subseteq \{\Sigma^* \mid |w| < k\}$ такие, что F содержит хотя бы одно слово длины k , которое не является надстрокой ни одного другого элемента F , и

$$\mathcal{L} = \left\{ w \mid w \notin W \wedge \forall u \in P, v \in S, f \in F, w_1, w_2 \in \Sigma^* \right. \\ \left. (w \neq uw_2 \wedge w \neq w_1v \wedge w \neq w_1fw_2) \right\}.$$

Если язык k -СЛТ для некоторого $k \in \mathbb{N}$, будем опускать k и говорить просто об СЛТ языке. Если автомат распознает СЛТ язык, будем говорить, что автомат СЛТ. Определить принадлежность слова w k -СЛТ языку можно с помощью скользящего окна длины k для слов длины k и более или непосредственно проверкой принадлежности w к W , если $|w| < k$. Определение, данное выше, несколько отличается от классического определения СЛТ языка [1], тем не менее, в случае, когда алфавит конечен, эти определения эквивалентны.

Определение 17 (Запрещенный инфикс). Слово w называется запрещенным инфиксом для языка L , если ни одно слово из $\{u w v \mid u, v \in \Sigma^*\}$ не принадлежит языку L .

По аналогии с запрещенным инфиксом, можно определить запрещенный префикс, суффикс и слово.

Определение 18 (Кратчайший запрещенный инфикс). Слово w называется кратчайшим запрещенным инфиксом для языка L , если w является запрещенным инфиксом для L и не существует собственного инфикса w' такого, что w' запрещен в L как инфикс. Будем обозначать множество запрещенных инфиксов для L как $SFF(L)$.

Определение 19 (Кратчайший запрещенный префикс (суффикс)). Слово w называется кратчайшим запрещенным префиксом (суффиксом) для языка L , если w является запрещенным префиксом (суффиксом) для L и не существует собственного префикса (суффикса) w' такого, что w' запрещен в L как префикс (суффикс). Кроме того, $\forall i \in SFF(L)$ $w \neq uiv$, где $u, v \in \Sigma^*$. Будем обозначать множество запрещенных префиксов (суффиксов) для L как $SFF(L)$ ($SFS(L)$).

Определение 20 (Кратчайшее запрещенное слово). Слово w называется кратчайшим запрещенным словом для языка L , если w является запрещенным словом для L и не существует собственного слова w' такого, что w' запрещен в L как слово. Кроме того, запрещенное слово не должно начинаться с запрещенного префикса, заканчиваться запрещенным суффиксом или содержать запрещенный инфикс. Будем обозначать множество запрещенных слов для L как $SFW(L)$.

Нормальной формой СЛТ языка L будем называть такое его представление, в котором $F = SFF(L)$, $P = SFP(L)$, $S = SFS(L)$, $W = SFW(L)$.

6.1 Получение нормальной формы СЛТ языка

Приведение к нормальной форме СЛТ языка можно осуществить с помощью алгоритма, описанного Яноушеком и Плахой [2]. Алгоритм принимает на вход ДКА, задающий некоторый язык L и строит по нему шаблонный автомат, выбирая различным образом начальное и финальные состояния в котором, можно получить автоматы, распознающие обращенные множества SFF^R , SFP^R , SFS^R , SFW^R .

Состояния шаблонного автомата есть подмножества множества состояний исходного ДКА, помеченные булевым флагом. Алгоритм строит функцию перехода шаблонного автомата, с помощью обхода искомого ДКА в ширину: для каждого состояния исходного ДКА и для каждой буквы рассматриваются состояния, из которых есть переход по этой букве в рассматриваемое состояние. Несмотря на временную сложность в худшем случае $O(3^{|Q|})$, алгоритм не обходит явно все $3^{|Q|}$ помеченных подмножеств, обрабатывая только достижимые состояния шаблонного автомата. Помимо этого, авторы используют некоторые эвристики для ускорения алгоритма на практике. Роджерс и Ламберт, использующие схожую конструкцию для получения нормальной формы СЛТ в своей работе [3], связанной с анализом фонетических акцентов в речи, отчитались о приемлемом времени

работы алгоритма на наборе данных. Авторы алгоритма указывают, что для ДКА, распознающих СЛТ языки, встречающихся на практике, размер шаблонного автомата пропорционален размеру множеств запрещенных инфиксов, префиксов, суффиксов и слов.

Более подробное описание алгоритма можно найти в работе [2], сейчас же рассмотрим пример, когда шаблонный автомат, построенный по СЛТ ДКА, асимптотически экспоненциально больше исходного ДКА.

Во избежание путаницы, будем обозначать элементы целочисленного алфавита жирным начертанием, например бинарный целочисленный алфавит будем обозначать как $\{0, 1\}$. Рассмотрим ДКА $A = \langle Q, \Sigma, \delta, q_0, F \rangle$ с множеством состояний $Q = \{q_0, q_1, q_2, \dots, q_n, q_{\text{АССЕРТ}}\}$ над целочисленным алфавитом $\Sigma = \{1, 2, 3, \dots, 2^n - 1, 2^n\}$. Помимо этого, введем над алфавитом $B = \{0, 1\}$ множество B^n всех строк длины n , обозначим его элементы $b_1, b_2, \dots, b_{2^n}$. Функция перехода δ ДКА A определяется следующим образом:

$$\begin{aligned} \forall d \in \{1, 2, \dots, n\} \quad \delta(q_0, d) &= q_d \\ \forall d \in \Sigma \quad \forall i \in \overline{1, n} \quad \delta(q_i, d) &= q_{\text{АССЕРТ}} \iff b_d[i] = 1 \end{aligned}$$

Графическая схема автомата A приведена на рисунке 4.

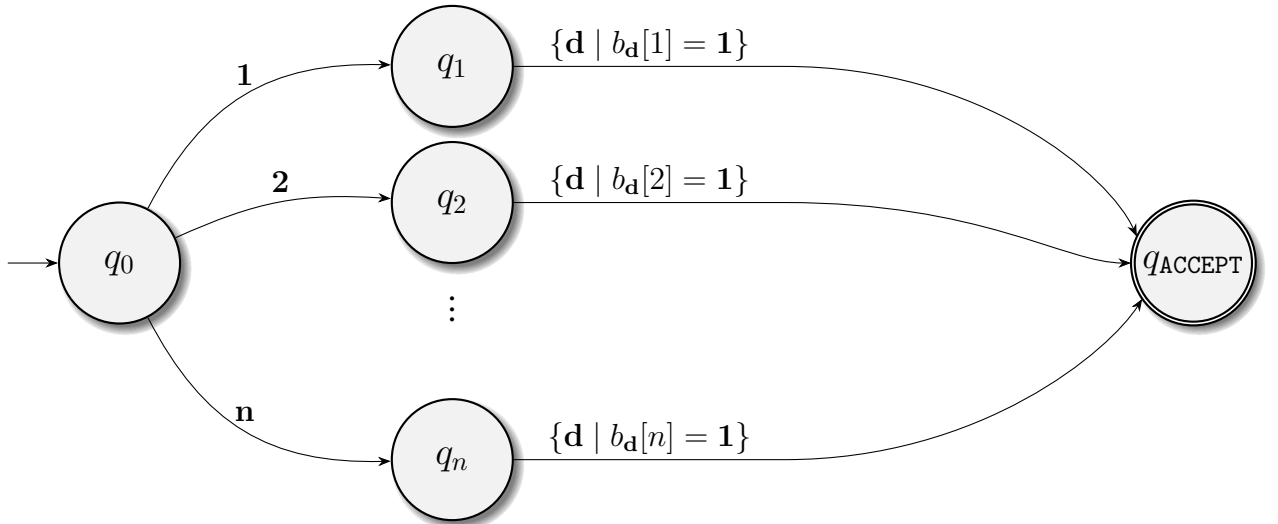


Рис. 4 — ДКА A , иллюстрирующий экспоненциальный рост шаблонного автомата. Переходы между q_i, q_j по разным буквам объединены в множество:
 $q_i \xrightarrow{D \subseteq \Sigma} q_j \stackrel{\text{def}}{\iff} \delta(q_i, d) = q_j \quad \forall d \in D$

Пусть $L = \mathcal{L}(A)$, так как L конечен, он является СЛТ языком. Рассмотрим язык $SFF^R(L)$. По построению очевидно, что язык L содержит только слова длины 2, тогда все его минимальные запрещенные инфиксы имеют длину от 1 до 3. Заметим, что множество символов, встречающихся в словах L совпадает с алфавитом Σ , значит минимальных запрещенных инффиксов длины 1 быть не может. Рассмотрим минимальные запрещенные инффиксы длины 2 языка L . В частности, рассмотрим запрещенные инффиксы, начинающиеся с $\mathbf{k} \in \{1, 2, \dots, n\}$. Искомый автомат распознает слова $\mathbf{k}\{\mathbf{d} \mid b_d[k] = 1\}$, тогда запрещенными инффиксами будут слова $\mathbf{k}\{\mathbf{d} \mid b_d[k] = 0\}$. Таким образом, все запрещенные инффиксы длины 2 имеют вид $FF_2 = \bigcup_{k=1}^n \mathbf{k}\{\mathbf{d} \mid b_d[k] = 0\}$. Очевидно, ни один элемент FF_2 не является подстрокой другого элемента FF_2 . Кроме того, если минимальных запрещенных инффиксов длины 1 быть не может, множество FF_2 есть минимальные запрещенные инффиксы длины 2. Рассмотрим множество $SFF_2^R = FF_2^R = \bigcup_{k=1}^n \{\mathbf{d} \mid b_d[k] = 0\}\mathbf{k}$.

Утверждение 1. Пусть ДКА A минимален для языка L , заданного конечным множесвтом слов длины k . Тогда минимальный ДКА для $L \cup L'$, $|w| > k \forall w \in L'$ имеет не меньше состояний, чем ДКА A .

Доказательство. Какое-нибудь очевидное доказательство того, что отношение Нероуда на $L \cup L'$ есть измельчение отношения Нероуда на L . $\square$

Из утверждения следует, что минимальный ДКА $A_{2 \cup 3} \mid \mathcal{L}(A_{2 \cup 3}) = FF_2 \cup FF_3$ содержит не меньше состояний, чем минимальный ДКА $A_2 \mid \mathcal{L}(A_2) = FF_2$. Аналогичное рассуждение справедливо для множеств SFF_2^R и SFF_3^R . Кроме того, $SFF_2 \cup SFF_3 = SFF(L) \implies SFF_2^R \cup SFF_3^R = SFF^R(L)$. Исходя из предыдущих утверждений, можно заключить, что число состояний ДКА, распознающего язык $SFF^R(L)$, не меньше, чем число состояний минимального ДКА $A_2^R \mid \mathcal{L}(A_2^R) = SFF_2^R$.

Утверждение 2. Для языка SFF_2^R для любых различных $\mathbf{d}_1, \mathbf{d}_2 \in \Sigma$ $[\mathbf{d}_1] \neq [\mathbf{d}_2]$.

Доказательство. Покажем от противного, что существует $d_3 \in \{1, 2, \dots, n\}$ такое, что

$$\mathbf{d}_1\mathbf{d}_3 \in L \wedge \mathbf{d}_2\mathbf{d}_3 \notin L \quad \vee \quad \mathbf{d}_1\mathbf{d}_3 \notin L \wedge \mathbf{d}_2\mathbf{d}_3 \in L.$$

Пусть $\mathbf{d}_3$ с необходимым свойством не существует, тогда для любого k слова $\mathbf{d}_2k$ и $\mathbf{d}_1k$ либо одновременно принадлежат L , либо одновременно не принадлежат L . Известно, что $SFF_2^R = \bigcup_{k=1}^n \{\mathbf{d} \mid b_{\mathbf{d}}[k] = 0\}k$. Тогда условие одновременной (не)принадлежности языку можно записать, как

$$b_{\mathbf{d}_1}[k] = b_{\mathbf{d}_2}[k] \forall k \in \{1, 2, \dots, n\} \implies b_{\mathbf{d}_1} = b_{\mathbf{d}_2} \text{ для различных } \mathbf{d}_2, \mathbf{d}_1.$$

Последнее, очевидно, невозможно в силу определения b_i как различных элементов множества $B = \{0, 1\}^k$. Итак, для букв $\mathbf{d}_1, \mathbf{d}_2$ существует правый контекст такой, что условие одновременной (не)принадлежности не выполняется. Следовательно $\mathbf{d}_1, \mathbf{d}_2$ принадлежат разным классам эквивалентности. В силу произвольного выбора $\mathbf{d}_1, \mathbf{d}_2$, утверждение доказано. $\square$

Число состояний в минимальном ДКА языка равно числу классов эквивалентности по отношению Нероуда на языке. Таким образом, число состояний в минимальном ДКА, распознающем язык SFF_2^R , больше или равно $|\Sigma| = 2^n$, следовательно, в минимальном ДКА языка SFF^R также будет не менее 2^n состояний. Напомним, что по построению исходный ДКА A содержит $n + 3$ состояний. Так как шаблонный атвومات для A не меньше, чем автомат для $SFF^R(L)$, число состояний в шаблонном автомате для A экспоненциально больше, чем число состояний исходного ДКА. Более формально, число состояний в шаблонном автомате $|Q_{TA}|$ для данного A с $|Q|$ состояниями оценивается как $\Omega(2^{|Q|})$.

6.2 Шаблоны, описываемые СЛТ языками

Не все шаблоны допускают описание с помощью СЛТЯ. Алгоритм приведения шаблона к регулярному виду всегда порождает регулярное выражение, являющееся конкатенацией либо повторения любого символа 0 и более раз, либо альтернативы с выбором из некоторого подмножества Γ_0 . Будем называть непрерывным конечным блоком альтернатив конкатенацию вида

$$(a_{11}|a_{12}|\dots|a_{1k_1})(a_{21}|a_{22}|\dots|a_{2k_2})\dots(a_{n1}|a_{n2}|\dots|a_{nk_n}).$$

Будем обозначать такие блоки как r^+ . Языком блока $r^+ = (a_{11}|a_{12}|\dots|a_{1k_1})(a_{21}|a_{22}|\dots|a_{2k_2})\dots(a_{n1}|a_{n2}|\dots|a_{nk_n})$ будем называть

$$\mathcal{L}(r^+) = \{b_1b_2\dots b_n \mid b_i \in \{a_{i1}|a_{i2}|\dots|a_{ik_i}\}\}$$

Тогда общий вид регулярного представления можно записать, как $\{r^+, .^*\}^*$.

Пусть $\Phi \in \mathcal{W}_{\text{pat}}$ — слово, не содержащее $-$ переменных на внешнем скобочном уровне. Тогда антиязыки шаблонов $\overline{\mathcal{L}(p)}$ из ниже описанных семейств допускают СЛТ описание:

$$\begin{aligned}\overline{\mathcal{L}(e\Phi e)} &\mapsto \overline{R(.^*r^+.^*)} \mapsto \langle \emptyset, \mathcal{L}(r^+), \emptyset, \emptyset \rangle, \\ \overline{\mathcal{L}(e\Phi)} &\mapsto \overline{R(.^*r^+)} \mapsto \langle \emptyset, \emptyset, \mathcal{L}(r^+), \emptyset \rangle, \\ \overline{\mathcal{L}(\Phi e)} &\mapsto \overline{R(r^+.^*)} \mapsto \langle \mathcal{L}(r^+), \emptyset, \emptyset, \emptyset \rangle, \\ \overline{\mathcal{L}(\Phi)} &\mapsto \overline{R(r^+)} \mapsto \langle \emptyset, \emptyset, \emptyset, \mathcal{L}(r^+) \rangle.\end{aligned}$$

Покажем, что, например, антиязык $e\Phi e\Phi$ уже не допускает СЛТ-представления. Рассмотрим шаблон $eses$ из семейства $e\Phi e\Phi$, докажем, что в алфавите $\{s, b\}$ его антиязык не допускает СЛТ представления. Для этого введем определение константы и критерий k -СЛТ свойства [4].

Определение 21. Если S — полугруппа и X — подмножество S , то элемент $c \in S$ называется константой относительно X , если для всех $p, q, r, s \in S$ $pcq, rcs \in X \implies pcs \in X$.

Теорема 1. Пусть L — формальный язык над алфавитом Σ . Пусть S — его синтаксический моноид. $\text{const}(S)$ — множество констант для S относительно $\text{im } \phi(L)$, ϕ — синтаксический морфизм, ставящий в соответствие элементам $w \in L$ элементы $[w] \in S$. $\text{idem}(S)$ — множество идемпотентов для S .

$$Lk - \text{СЛТ} \iff ([w] \in \text{idem}(S) \implies [w] \in \text{const}(S))$$

Теорема сформулирована де Лукой и Рестиво [4], доказательство теоремы можно найти там же.

Синтаксический моноид языка $L = \overline{\mathcal{L}(eses)}$ над b, s есть $S = \{[b], [s], [ss], [sb]\}$, при этом $\text{im } \phi(L) = \{[b], [s], [sb]\}$.

Утверждение 3. Идемпотент $[b]$ не является константой относительно $\{[b], [s], [sb]\}$.

Доказательство. Пусть $p = s = [s]$, $q = r = [b]$

$$pcq = [s][b][b] = [sbb] = [sb] \in \text{im } \phi(L)$$

$$rcs = [b][b][s] = [bbs] = [s] \in \text{im } \phi(L)$$

$$pcs = [s][b][s] = [sbs] = [ss] \notin \text{im } \phi(L)$$

□

Итак, получили, что $[b] \in \text{idem}(S) \wedge [b] \notin \text{const}(S)$, следовательно, искомый язык $\overline{\mathcal{L}(eses)}$ не является СЛТ.

7 Распознавание перекрытий на основе антиязыков шаблонов

Для шаблонов из семейств, описанных выше, можно эффективно решать упрощенную задачу перекрытия, когда требуется определить наличие перекрытия и предоставить хоть какое-то множество индексов предложений, перекрывающих другое предложение.

Алгоритм основывается на двух фактах:

1. СЛТ языки замкнуты относительно пересечения.
2. Антиязык предложения есть множество выражений, которые “проваливаются” вниз (т.к. сопоставление проводится сверху вниз).

Т.е. до предложения $f.p_i$ “доходят” только выражения из множества $\bigcap_{j=1}^{i-1} \overline{\mathcal{L}(f.p_j)}$. Алгоритм извлечения аннотированного ДКА позволяет извлекать ДКА, распознающие конкретный шаблон соответствующего уровня вложенности. Если шаблон допускает СЛТ представление, то его нормальную СЛТ форму можно определить с помощью алгоритма из работы [2]. Для двух СЛТ языков L и L' , представленных в нормальной форме $\langle P, F, S, W \rangle$ и $\langle P', F', S', W' \rangle$, можно эффективно проверять их на включение: L включает L' тогда и только тогда, когда каждый префикс $p \in P$ является префиксом некоторого префикса $p' \in P'$ или содержит инфикс u_p такой, что $u_p \in F'$, каждый инфикс $f \in F$ является инфиксом некоторого инфикса $f' \in F'$, каждый суффикс $s \in S$ является суффиксом некоторого суффикса $s' \in S'$ или содержит инфикс u_s такой, что $u_s \in F'$, каждое слово $w \in W$ запрещено в L' . Построив автомат Ахо-Корасик для F' и автоматы, распознающие P', S', W' , можно эффективно проверять включение L в L' за $\mathcal{O}(\max(|P|, |P'|), \max(|S|, |S'|), \max(|F|, |F'|), \max(|W|, |W'|))$ времени, просто проверяя условия выше прогонкой элементов из P, F, S, W по различным автоматам для P', F', S', W' .

Нахождение пересечения двух СЛТ языков также реализуется сравнительно нетрудно — достаточно объединить множества P, F, S, W для L и P', F', S' для L' и найти нормальную СЛТ форму для L'' , заданного $\langle P \cup P', F \cup F', S \cup S', W \cup W' \rangle$.

8 Результат научно-исследовательской работы

В ходе работы были проведен анализ работ, посвященных СЛТ-языкам и их свойствам. Разработан алгоритм распознавания перекрытий на основе антиязыков шаблонов на основе свойств СЛТ-языков.

Использование регулярного представления абстрактных шаблонов можно использовать и с целью порождения документации: например, для функции, последним предложением которой является шаблон $e \cdot _$, с помощью описания множества выражений, которые “проваливаются” к последнему предложению, можно сгенерировать описание выражений, которые могут быть распознаны последним предложением. Очевидно, что в общем случае, это почти всегда не будет совпадать с множеством $\mathcal{W}_{\text{expr}}$, которое сопоставляется $e \cdot _$ при наивном анализе.

Помимо этого, класс рассматриваемых шаблонов можно расширить, избавившись от ограничения на строгую локальную тестируемость антиязыка. Любое регулярное представление языка и антиязыка абстрактного шаблона без повторных переменных будет принадлежать к классу кусочно тестируемых языков. Анализ таких языков в общем случае значительно сложнее, поэтому не рассматривается в данной работе.

Список литературы

1. McNaughton R., Papert S. Counter-Free Automata. — Cambridge, MA : MIT Press, 1971.
2. Janoušek J., Plachý Š. Shortest characteristic factors of a deterministic finite automaton and computing its positive position run by pattern set matching // Acta Informatica. — 2025. — T. 62, № 36. — DOI: 10.1007/s00236-025-00484-0.
3. Rogers J., Lambert D. Extracting Subregular constraints from Regular stringsets // Journal of Language Modelling. — 2019. — T. 7, № 2. — С. 143—176.
4. De Luca A., Restivo A. A Characterization of Strictly Locally Testable Languages and Its Application to Subsemigroups of a Free Semigroup // Information and Control. — 1980. — T. 44, № 3. — С. 300—319. — DOI: 10.1016/S0019-9958(80)90150-8.